Blok C (80 min): Python pakiranje

> 🎯 Ishod vježbe
>
> U osobnom repozitoriju `praktikum-<github-korisnik>` otvoren je **pull request** s grane `feature/python-package`. PR sadrži:
>
> - Python **paket** `praktikum_llm` u `src/` (umjesto modula `llm.py` i `ollama_client.py` u korijenu, premještenih s `git mv`),
> - `pyproject.toml` s ovisnostima, `[dev]` ovisnostima, naredbom `llm` i verzijom iz `version.txt`,
> - testove prilagođene paketu (`tests/test_client.py`, `tests/test_version.py`),
> - `Dockerfile` s fazama `build` (wheel), `test` i `runtime`,
> - `docker.txt` s izlazom provjera iz koraka 5.
>
> **Kriteriji prihvaćanja** (svi moraju vrijediti, nastavnik ih provjerava u PR-u):
>
> 1. Lokalno, nakon `pip install -e ".[dev]"`: `python -m pytest` završava s `6 passed`.
> 2. Lokalno: `cd /tmp && llm --version` ispisuje verziju iz `version.txt` (naredba radi iz bilo kojeg direktorija).
> 3. `docker build --target test .` završava s `6 passed`.
> 4. `docker run --rm praktikum-llm --version` ispisuje istu verziju, a `pip show praktikum-llm` u slici pokazuje instalirani paket.
> 5. Runtime slika nema izvorni kod: `docker run --rm --entrypoint ls praktikum-llm /app /src` javlja da direktoriji ne postoje.
> 6. Poruka završnog commita i naslov PR-a slijede Conventional Commits i povećavaju **minor** verziju.
>
> **Predaja:** otvoren PR. Ne spajati ga sami; PR je predaja i ostaje otvoren za pregled.

Polazište je rezultat bloka B (grana `feature/docker`). Gotovo rješenje nalazi se u docker-lab/blok-c/.

Sadržaj

0. Grana

1. Modul i paket: u čemu je razlika

2. Pretvorba modula u paket

3. Dockerfile s build fazom

4. Što pakiranje mijenja u deploymentu

5. Provjere, commit i pull request

0. Grana

```
cd ~/praktikum-$(gh api user -q .login)
git fetch origin
git checkout -b feature/python-package origin/feature/docker   # or
origin/main if the block B PR is already merged
```

Ako PR iz bloka B još nije spojen, PR iz ovog bloka otvaramo prema grani `feature/docker` (vidi korak 5), pa diff pokazuje samo pakiranje.

1. Modul i paket: u čemu je razlika

Pojam	Što je	Primjer
Modul	jedna `.py` datoteka; ime modula = ime datoteke	`ollama_client.py` → `import ollama_client`
Paket (import package)	direktorij modula s `__init__.py`; ima vlastiti imenski prostor	`src/praktikum_llm/` → `from praktikum_llm import client`
Distribucijski paket	ono što `pip` instalira: ime, verzija, ovisnosti, ulazne točke	`praktikum-llm` u `pyproject.toml`
Wheel (`.whl`)	gotova, verzionirana arhiva distribucijskog paketa	`praktikum_llm-0.1.0-py3-none-any.whl`

Do sada `import ollama_client` (i `import llm` u testovima) radi samo zato što Python pokrećemo **iz korijena repozitorija** (trenutni direktorij je na `sys.path`). Ime `ollama_client` je globalno: bilo koji drugi `ollama_client.py` na putu može ga zasjeniti. Paket daje imenski prostor (`praktikum_llm.client`), metapodatke (ime, verzija, ovisnosti) i instalaciju koja ne ovisi o tome gdje stojimo.

2. Pretvorba modula u paket

Ciljna struktura (*src layout*):

```
praktikum-<github-korisnik>/
├── pyproject.toml
├── version.txt                # release-please (week 1) keeps bumping it
├── Dockerfile                 # from block B, changed in step 3
├── .dockerignore
├── src/
│   └── praktikum_llm/
│       ├── __init__.py        # package marker + __version__
│       ├── __main__.py        # python -m praktikum_llm
│       ├── cli.py             # was llm.py
│       └── client.py          # was ollama_client.py
└── tests/
    ├── test_client.py         # was tests/test_ollama_client.py
    └── test_version.py        # rewritten for the package
```

`git mv` čuva povijest datoteka (`git log --follow src/praktikum_llm/client.py`):

```
mkdir -p src/praktikum_llm
git mv ollama_client.py src/praktikum_llm/client.py
git mv llm.py src/praktikum_llm/cli.py
git mv tests/test_ollama_client.py tests/test_client.py
git rm requirements.txt requirements-dev.txt   # dependencies now live in
pyproject.toml
```

`pyproject.toml` (gotova datoteka):

```
[build-system]
requires = ["setuptools>=69"]
build-backend = "setuptools.build_meta"

[project]
name = "praktikum-llm"
dynamic = ["version"]
description = "Command line client for a local Ollama model (Praktikum:
Razvojne operacije)."
requires-python = ">=3.10"
dependencies = [
    "requests>=2.31",
    "python-dotenv>=1.0",
]

[project.optional-dependencies]
dev = ["pytest>=8"]

[project.scripts]
llm = "praktikum_llm.cli:main"

[tool.setuptools.dynamic]
# release-please (week 1) bumps version.txt; the package version follows
it.
version = { file = "version.txt" }

[tool.pytest.ini_options]
testpaths = ["tests"]
```

`src/praktikum_llm/__init__.py`:

```
"""Command line client for a local Ollama model."""
from importlib.metadata import PackageNotFoundError, version

try:
    __version__ = version("praktikum-llm")
except PackageNotFoundError:  # running from a source tree that was never
```

```
installed
    __version__ = "0.0.0"
```

src/praktikum_llm/__main__.py:

```
"""Allows `python -m praktikum_llm ...` in addition to the `llm`
command."""
import sys

from praktikum_llm.cli import main

sys.exit(main())
```

Izmjene u `client.py` (bivši `ollama_client.py`): obrisati `from dotenv import load_dotenv` i `load_dotenv()`. Biblioteka ne čita `.env` pri importu; to radi CLI. Zadane vrijednosti su u kodu od bloka B. **Izmjene u `cli.py`** (bivši `llm.py`), cijela datoteka u docker-lab/blok-c/src/praktikum_llm/cli.py:

```
import signal
from dotenv import find_dotenv, load_dotenv

from praktikum_llm import __version__, client     # was: import
ollama_client
```

- `ollama_client.` → `client.` u cijeloj datoteci;
- obrisati `VERSION_FILE` i `read_version()`; u `build_parser()` verzija dolazi iz metapodataka paketa: `version=f"%(prog)s {__version__}"`;
- obrisati blok `if __name__ == "__main__":` (naredbu `llm` sada stvara `pip` iz `[project.scripts]`);
- handler za SIGTERM ostaje, a `.env` se učitava u `main()`:

```
def main() -> int:
    signal.signal(signal.SIGTERM, _exit_on_sigterm)
    # usecwd=True: look for .env in the current directory, not next to the
installed package.
    load_dotenv(find_dotenv(usecwd=True))
    args = build_parser().parse_args()
    ...
```

Zašto `usecwd=True`? Bez njega `find_dotenv()` traži `.env` počevši od direktorija datoteke koja ga poziva. Kad je paket instaliran, to je `site-packages`, pa se `.env` iz projekta tiho ne učita.

Zašto `read_version()` više ne valja? `Path(__file__).with_name("version.txt")` traži datoteku pokraj modula. Kad je alat instaliran kao paket, modul je u `site-packages/praktikum_llm/`, a `version.txt` nije dio wheela, pa bi `llm --version` tiho javio `0.0.0`. U paketu verziju čita alat za

build iz `version.txt` (`[tool.setuptools.dynamic]`) i zapisuje je u metapodatke; `importlib.metadata.version("praktikum-llm")` je čita odande, gdje god je paket instaliran. Posljedica: verzija se "zamrzne" u trenutku instalacije. Promjena `version.txt` vidi se tek nakon ponovnog `pip install`.

Testovi: u `tests/test_client.py` zamijeniti `import ollama_client` s `from praktikum_llm import client`, a `ollama_client.` s `client.`. `tests/test_version.py` sada provjerava metapodatke paketa:

```
"""The installed package reports the version from version.txt (read once,
at build time)."""
from pathlib import Path

import praktikum_llm

VERSION_FILE = Path(__file__).resolve().parent.parent / "version.txt"


def test_package_version_matches_version_file():
    assert praktikum_llm.__version__ ==
VERSION_FILE.read_text(encoding="utf-8").strip()
```

`.gitignore`: dodati `build/`, `dist/` i `*.egg-info/`.

Instalacija u venv i provjera:

```
source .venv/bin/activate
pip install -e ".[dev]"          # editable: changes in src/ are visible
without reinstalling
python -m pytest -v              # 6 passed
llm --version                    # llm 0.0.x, from version.txt
llm status
python -m praktikum_llm models
which llm                        # .venv/bin/llm, generated by pip
cd /tmp && llm --version && cd - # no python, no path: the command works
anywhere
```

3. Dockerfile s build fazom

U bloku B faza `base` instalirala je ovisnosti, a kod smo kopirali kao datoteke. Sada imamo pravi artefakt koji treba izgraditi: wheel. Zato se vraća faza `build`, a `runtime` instalira samo wheel (nema izvornog koda ni `requirements*.txt`). Zamijeniti `Dockerfile` (gotova datoteka):

```
# syntax=docker/dockerfile:1
# Multi-stage build of the praktikum-llm package.
#   docker build -t praktikum-llm .               -> runtime image
(default, last stage)
```

```
#   docker build --target test -t llm-test .      -> runs the tests during
the build

ARG PYTHON_IMAGE=python:3.13-alpine

# ---- build: turn the source tree into a wheel ----
FROM ${PYTHON_IMAGE} AS build
WORKDIR /src
RUN pip install --no-cache-dir build
COPY pyproject.toml version.txt ./
COPY src/ src/
RUN python -m build --wheel --outdir /dist

# ---- test: install the wheel with dev extras and run pytest against it --
--
FROM ${PYTHON_IMAGE} AS test
WORKDIR /src
COPY --from=build /dist/ /dist/
RUN pip install --no-cache-dir "$(ls /dist/*.whl)[dev]"
COPY pyproject.toml version.txt ./
COPY tests/ tests/
RUN python -m pytest -v

# ---- runtime: only the installed package, running as a non-root user ----
FROM ${PYTHON_IMAGE} AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
RUN adduser -D -u 10001 app
COPY --from=build /dist/ /tmp/dist/
RUN pip install --no-cache-dir /tmp/dist/*.whl && rm -rf /tmp/dist
USER app
WORKDIR /home/app
ENTRYPOINT ["llm"]
CMD ["--help"]
```

U `.dockerignore` dodati `build/`, `dist/`, `*.egg-info` i promijeniti `__pycache__/*.pyc` u `**/__pycache__/**/*.pyc` (sada su u poddirektorijima).

```
docker build --target test -t llm-test .      # 6 passed, against the
installed wheel
docker build -t praktikum-llm .               # BuildKit skips the test
stage: runtime does not depend on it
docker images praktikum-llm                   # compare with the block B
image
docker run --rm praktikum-llm --version
docker run --rm --entrypoint id praktikum-llm
docker run --rm --entrypoint sh praktikum-llm -c 'ls /app /src; which llm;
pip show praktikum-llm | head -2'   # no source code in the image
```

Pokretanje s Ollamom je isto kao u bloku B; mijenja se samo `ENTRYPOINT` (`llm` umjesto `python llm.py`):

```
docker run --rm --network host praktikum-llm status                    # Docker Engine in WSL
docker run --rm -e OLLAMA_BASE_URL=http://host.docker.internal:11434 praktikum-llm status   # Docker Desktop
docker run --rm -it --network host praktikum-llm chat
```

4. Što pakiranje mijenja u deploymentu

	Moduli u korijenu (blok B)	**Paket (`pyproject.toml` + wheel)**
Što isporučujemo	izvorni kod + `requirements.txt`	jedan verzionirani artefakt: `praktikum_llm-0.0.1-py3-none-any.whl`
Pokretanje	`python llm.py` iz točnog direktorija	`llm` s bilo kojeg mjesta (`pip` stvara naredbu) ili `python -m praktikum_llm`
Importi	ovise o trenutnom direktoriju i `sys.path`	imenski prostor `praktikum_llm`, isto u venvu, slici i na VM-u
Ovisnosti	dvije `requirements*.txt` datoteke s točnim verzijama	`dependencies` (rasponi) za runtime, `[dev]` za razvoj i testove; točne verzije po potrebi u lock datoteci
Verzija	`read_version()` čita `version.txt` pokraj koda, u trenutku pokretanja	upisana u metapodatke pri buildu; `llm --version`, `pip show`, `importlib.metadata`
Docker slika	`base` + kopirane `.py` datoteke, `WORKDIR` mora biti točan	`build` napravi wheel, `runtime` ga samo instalira; bez izvornog koda i alata za build
Testovi	testiraju datoteke u radnom direktoriju	testiraju **instalirani** paket (src layout), kao u produkciji
Distribucija	`git clone`	PyPI, privatni registar (GCP Artifact Registry ima i Python repozitorije), `pip install git+https://...`

Ključna ideja: **build once, deploy many**. Isti wheel (ili ista slika s istim tagom) ide u dev, test i prod; mijenja se samo konfiguracija iz okoline (tema 11). Cijena: potreban je korak builda i `pyproject.toml` se mora održavati.

5. Provjere, commit i pull request

```
{
  echo "## local tests";  python -m pytest -q 2>&1 | tail -1
  echo "## local version"; (cd /tmp && llm --version)
```

```
  echo "## docker build --target test"; docker build --target test --
progress plain --no-cache -t llm-test . 2>&1 | grep -E "passed|failed"
  echo "## image version"; docker run --rm praktikum-llm --version
  echo "## pip show";      docker run --rm --entrypoint pip praktikum-llm
show praktikum-llm 2>/dev/null | head -2
  echo "## no source";     docker run --rm --entrypoint ls praktikum-llm
/app /src 2>&1
} | tee docker.txt
```

Poruka završnog commita. Završni commit mora prema specifikaciji Conventional Commits povećati **minor** verziju (`x.Y.z`): kad se PR spoji u `main`, release-please (tjedan 1) otvara release PR s novom minor verzijom u `version.txt`. Koji tip (prefiks) poruke to postiže, pronađite sami u specifikaciji; u naredbama ispod umjesto njega piše `<type>`. Isti prefiks koristite i u naslovu PR-a.

```
git add -A
git status                        # .env, .venv, dist/, build/, *.egg-info
must NOT be listed
git commit -m "<type>: package the CLI as praktikum_llm"
git push -u origin HEAD
BASE=feature/docker               # use main if the block B PR is already
merged
gh pr create --base "$BASE" --title "<type>: package the CLI as
praktikum_llm" --body "Week 3, block C. Checks: see docker.txt"
```

Kad nastavnik spoji PR iz bloka B, GitHub ovaj PR automatski preusmjeri na `main` (ako se grana `feature/docker` pri spajanju obriše) ili ga treba ručno preusmjeriti: `gh pr edit --base main`.

Predaja (praktični zadatak 3, dio C): otvoren PR iz `feature/python-package` s datotekama iz ishoda i `docker.txt` koji potvrđuje kriterije 1–5 (kriterij 6 provjerava se iz poruke commita i naslova PR-a). PR ne spajati sami.

Česti problemi

Simptom	Rješenje
`ModuleNotFoundError: No module named 'praktikum_llm'` u testovima	Paket nije instaliran u venv: `pip install -e ".[dev]"`
`llm: command not found`	venv nije aktivan ili paket nije instaliran; `which llm`
`llm --version` javlja staru verziju	Verzija se upisuje pri instalaciji: nakon promjene `version.txt` ponoviti `pip install -e ".[dev]"`
`llm --version` javlja `0.0.0`	Ostao je stari `read_version()`; verzija dolazi iz `praktikum_llm.__version__`
`.env` se ne učitava kad se pokrene `llm`	`load_dotenv()` bez `find_dotenv(usecwd=True)` traži

Simptom	Rješenje
	`.env` u `site-packages`
`error: Multiple top-level packages discovered`	Paket nije u `src/`, nego u korijenu uz druge direktorije; držati se src layouta
`test_package_version_matches_version_file` pada u Dockeru	U fazi `test` nedostaje `COPY pyproject.toml version.txt ./`